

DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT

Course Code & Name : CSA0612 - Design and Analysis of Algorithms

Assignment Title: Efficient Route Planning Using Graph Algorithms

Course Outcome(s) – CO:

CO1: Analyse the efficiency of recursive and non-recursive algorithms using asymptotic notations (Big-O, Ω , Θ) and complexity-analysis methods.

CO2: Apply brute-force techniques such as sorting and searching to solve computational problems.

CO3: Apply divide-and-conquer and graph-based techniques to design efficient algorithmic solutions.

Bloom's Taxonomy Level L4 – Analyse; L5 – Evaluate; L6 – Create

SDG Mapping (SDG 1-17) SDG 9 – Industry, Innovation and Infrastructure

Problem Statement : A logistics company needs to determine the most efficient routes for transporting goods between multiple cities. The transportation network is represented as a weighted graph, where vertices represent cities and edges represent roads with associated travel costs or distances. Develop, implement, and evaluate suitable graph-based algorithms to determine the shortest paths between locations. Implement (i) Dijkstra's algorithm for graphs with non-negative edge weights and (ii) Bellman-Ford algorithm as an alternative shortest-path approach. Analyse the theoretical efficiency of both algorithms using Big-O, Ω , and Θ notations, including their time and space complexities. Experimentally evaluate both algorithms using graphs of increasing numbers of vertices and edges, considering sparse and dense networks. Measure execution time, number of edge relaxations/comparisons, memory utilisation, and scalability. Based on the theoretical and experimental results, identify the most appropriate algorithm for different transportation-network scenarios and justify the choice with respect to computational efficiency, graph characteristics, and practical constraints — reflecting Bloom's Levels 4–6 (Analyse, Evaluate, Create) and contributing to SDG 9 (Industry, Innovation and Infrastructure).

Constraints: Use a publicly available or synthetically generated weighted graph dataset with sufficient records to test multiple graph sizes. Edge weights must be non-negative for Dijkstra's algorithm; Bellman-Ford may additionally be tested with negative edges where appropriate, but negative cycles must be identified and reported. Both algorithms must be executed in the same computational environment using identical input graphs for fair comparison. Test both sparse and dense graphs and report empirical timings, edge-relaxation/comparison counts, memory utilisation, and scalability. Clearly document the graph-generation method or dataset source, assumptions, algorithm limitations, and the criteria used to recommend an algorithm.

Presented BY:

Manoj M(192511060),

Theshon Mishmaan

M(192572106),

Yokesh (192424174) ,

Jaya Krishna (192421259)

Dinesh (192424016),

Suggested Assessment Requirements

Component	Requirement
Algorithm Implementation	Implement Dijkstra's and Bellman-Ford algorithms.
Complexity Analysis	Derive and compare Big-O, Ω , and Θ time/space complexities.
Experimental Evaluation	Test increasing graph sizes and sparse/dense graph structures.
Performance Metrics	Record execution time, edge relaxations/comparisons, memory use, and scalability.
Evaluation & Recommendation	Recommend the suitable algorithm for different network conditions with justification.

Table of Contents

S. No.	Chapter / Section	Page No.
1	Problem Statement and Problem Formulation	6
2	Objective and Expected Outcomes	6
3	Requirements, Constraints and Assumptions	7
4	Application of Relevant Course Knowledge / Concepts	7
5	Design / Proposed Solution / Methodology	8
6	Algorithm / Pseudocode / Flowchart	8
7	Implementation / Source Code and Environment / Tools Used	9
8	Test Cases and Expected/Actual Results	9
9	Execution Screenshots / Outputs	10
10	Results and Validation	13

11	Analysis, Comparison, Trade-offs and Justification of the Final Solution	15
12	Broader Considerations / SDG Relevance	17
13	Conclusion, Limitations and Possible Improvements	20
14	Individual Contribution of Group Members	23
15	References	--
16	Appendix – One-Page Individual Reflection	--

. Problem Statement and Problem Formulation

A logistics company operates a transportation network connecting multiple cities and needs to determine the most efficient (minimum-cost) routes for moving goods between locations. The road network is naturally modelled as a weighted graph $G = (V, E)$, where each vertex $v \in V$ represents a city/depot and each edge $(u, v) \in E$ represents a direct road connecting two cities, weighted by a non-negative or, in special cases, negative cost value $w(u, v)$ representing distance, travel time, or fuel/toll cost.

The core computational problem is the Single-Source Shortest Path (SSSP) problem: given a source vertex s , determine the minimum-cost path from s to every other vertex in the graph. This assignment formulates the problem as follows:

- Input: A weighted, directed (or undirected) graph $G = (V, E, w)$ with $|V| = n$ vertices and $|E| = m$ edges, and a designated source vertex s .
- Output: For every vertex $v \in V$, the shortest-path distance $\text{dist}(s, v)$ and the corresponding path (sequence of cities/edges).
- Goal: Solve this efficiently for graphs of increasing size (sparse and dense), using two classical approaches — Dijkstra's algorithm and the Bellman-Ford algorithm — and compare their theoretical and empirical performance to recommend the most suitable algorithm for different network scenarios.

2. Objective and Expected Outcomes

Objectives:

- To design and implement Dijkstra's algorithm for shortest-path computation on graphs with non-negative edge weights.
- To design and implement the Bellman-Ford algorithm as an alternative approach that also handles negative edge weights and detects negative-weight cycles.
- To analyse the theoretical time and space complexity of both algorithms using Big-O, Big-Omega (Ω), and Big-Theta (Θ) notation.
- To experimentally evaluate both algorithms on synthetically generated graphs of varying size and density (sparse vs. dense).
- To measure and compare execution time, number of edge relaxations/comparisons, and memory utilisation.
- To recommend the most appropriate shortest-path algorithm for realistic transportation-network scenarios, justifying the choice with theoretical and empirical evidence.

Expected Outcomes:

- A working implementation of both algorithms validated against known shortest-path results.
- Quantitative performance data (tables/graphs) showing how each algorithm scales with graph size and density.

- A clear, justified recommendation of which algorithm to use for sparse vs. dense, and non-negative vs. negative-weight, transportation networks.
- Demonstration of Bloom's Taxonomy levels L4 (Analyse), L5 (Evaluate) and L6 (Create) through complexity analysis, comparative evaluation, and original implementation/design.

3. Requirements, Constraints and Assumptions

Requirements:

- A synthetic or publicly available weighted-graph dataset representing a road/logistics network, with graphs of at least 3–4 different sizes (e.g., 10, 50, 200, 1000 vertices).
- A programming environment capable of graph representation (adjacency list/matrix), timing instrumentation, and plotting (e.g., Python with NetworkX, NumPy, Matplotlib, and the time/tracemalloc modules).
- Both sparse ($m \approx n$) and dense ($m \approx n^2$) graph variants for each vertex count.

Constraints:

- Edge weights must be non-negative for Dijkstra's algorithm; Bellman-Ford may additionally be tested with negative edges, but any negative-weight cycle must be detected and reported rather than silently producing an incorrect result.
- Both algorithms must run on identical input graphs, in the same runtime environment and on the same hardware, to ensure a fair timing comparison.
- Execution time and memory measurements must exclude one-time setup costs (graph loading) and isolate only the shortest-path computation.

Assumptions:

- The transportation network is modelled as a static graph (edge weights/costs do not change during a single run).
- The graph is connected, or shortest paths to unreachable vertices are reported as infinity (∞).
- Edge weights represent a real, physically meaningful metric (distance in km, or cost in currency units).

- For Dijkstra's algorithm, a binary-heap (priority-queue) based implementation is used unless otherwise stated.

4. Application of Relevant Course Knowledge / Concepts

This assignment directly applies concepts covered under the Design and Analysis of Algorithms course, mapped to the following Course Outcomes (CO):

Concept	Course Outcome	Application in this Assignment
Asymptotic notation (Big-O, Ω , Θ)	CO1	Used to formally express the worst-case, best-case and tight-bound running times of Dijkstra's and Bellman-Ford's algorithms.
Greedy technique	CO2 / CO3	Dijkstra's algorithm is a classic greedy algorithm — it always selects the unvisited vertex with the smallest tentative distance.
Dynamic-programming-style relaxation	CO3	Bellman-Ford repeatedly relaxes all edges, building on previously computed sub-optimal distances (principle of optimality).
Graph representation & traversal	CO3	Adjacency list representation is used for efficient neighbour iteration in both algorithms.
Priority queues / heaps	CO1 / CO3	A min-heap is used to efficiently extract the minimum-distance vertex in Dijkstra's algorithm, reducing complexity from $O(V^2)$ to $O((V+E) \log V)$.
Empirical algorithm	CO1	Execution-time and

analysis

operation-count

measurements validate the
theoretical asymptotic
analysis experimentally.

5. Design / Proposed Solution / Methodology

The proposed solution follows a structured experimental methodology, illustrated below:

1. Graph Generation: Generate synthetic weighted graphs representing city networks of varying sizes (e.g., $n = 10, 50, 100, 500, 1000$ vertices) and densities (sparse: $m \approx 2n$; dense: $m \approx n(n-1)/2$).
2. Graph Representation: Store each graph internally as a weighted adjacency list for memory efficiency and fast neighbour lookup.
3. Algorithm Implementation: Implement Dijkstra's algorithm using a binary min-heap priority queue, and Bellman-Ford using iterative edge relaxation over $V-1$ rounds plus a final negative-cycle check.
4. Correctness Validation: Compare outputs of both algorithms against each other (and, for small graphs, against manually computed shortest paths) to confirm correctness.
5. Instrumentation: Add counters for edge relaxations/comparisons and wrap execution with high-resolution timers; use memory-profiling utilities to record peak memory usage.
6. Experimentation: Run both algorithms on every generated graph, repeating each run multiple times and averaging to reduce noise.
7. Analysis: Tabulate and plot execution time, relaxation counts and memory usage against graph size/density; compare against theoretical complexity curves.
8. Recommendation: Synthesise theoretical and empirical results into a justified recommendation for algorithm choice per network scenario.

System Architecture (high-level flow): Dataset/Graph Generator → Graph Representation (Adjacency List) → Algorithm Module (Dijkstra / Bellman-Ford) → Performance Logger (time, relaxations, memory) → Results Aggregator → Visualisation & Analysis (tables/graphs) → Recommendation.

6. Algorithm / Pseudocode / Flowchart

6.1 Dijkstra's Algorithm

```
DIJKSTRA(Graph G, source s):  
  for each vertex v in G.V:  
    dist[v] = INFINITY  
    prev[v] = NIL  
  dist[s] = 0  
  PriorityQueue Q // min-heap keyed on dist  
  Q.insert(s, 0)  
  
  while Q is not empty:  
    u = Q.extractMin()  
    for each neighbour v of u with weight w(u, v):  
      if dist[u] + w(u, v) < dist[v]:  
        dist[v] = dist[u] + w(u, v)  
        prev[v] = u  
        Q.decreaseKeyOrInsert(v, dist[v])  
  return dist, prev
```

6.2 Bellman-Ford Algorithm

```
BELLMAN_FORD(Graph G, source s):  
  for each vertex v in G.V:  
    dist[v] = INFINITY  
    prev[v] = NIL  
  dist[s] = 0  
  
  for i = 1 to |V| - 1:  
    for each edge (u, v) with weight w(u, v) in G.E:  
      if dist[u] + w(u, v) < dist[v]:
```

```

        dist[v] = dist[u] + w(u, v)
        prev[v] = u

// Negative-cycle detection
for each edge (u, v) with weight w(u, v) in G.E:
    if dist[u] + w(u, v) < dist[v]:
        report "Negative-weight cycle detected"
        return FAILURE

return dist, prev

```

6.3 Flowchart (Textual Representation)

Start → Read Graph & Source → Initialise distances to ∞ (source = 0) → [Dijkstra: pick min-distance unvisited vertex via heap | Bellman-Ford: relax all edges] → Relax adjacent edges / update distances → Repeat until Queue empty (Dijkstra) or $V-1$ iterations complete (Bellman-Ford) → (Bellman-Ford only) Check for negative cycle → Output shortest distances and paths → End.

Note: For the final report, this textual flowchart should be redrawn as an actual flowchart diagram (e.g., using draw.io, Lucidchart, or PowerPoint SmartArt) and inserted as an image here.

7. Implementation / Source Code and Environment / Tools Used

Environment / Tools Used:

- Language: Python 3.x
- Libraries: heapq (priority queue for Dijkstra), NetworkX (graph generation & validation), NumPy, Matplotlib (plots), time / timeit (execution-time measurement), tracemalloc (memory profiling)
- IDE/Platform: VS Code / Jupyter Notebook / Google Colab
- Version Control: Git & GitHub for source code management

Sample Implementation (Dijkstra's Algorithm in Python):

```
import heapq

def dijkstra(graph, source):
    dist = {v: float('inf') for v in graph}
    dist[source] = 0
    visited = set()
    pq = [(0, source)]
    relaxations = 0

    while pq:
        d, u = heapq.heappop(pq)
        if u in visited:
            continue
        visited.add(u)
        for v, w in graph[u]:
            relaxations += 1
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heapq.heappush(pq, (dist[v], v))
    return dist, relaxations
```

Sample Implementation (Bellman-Ford Algorithm in Python):

```
def bellman_ford(vertices, edges, source):
    dist = {v: float('inf') for v in vertices}
    dist[source] = 0
    relaxations = 0

    for _ in range(len(vertices) - 1):
        for u, v, w in edges:
            relaxations += 1
```

```

        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w

    for u, v, w in edges:
        if dist[u] + w < dist[v]:
            raise ValueError("Graph contains a negative-weight cycle")

    return dist, relaxations

```

The complete, runnable source code (graph generator, both algorithms, instrumentation, and plotting scripts) should be included in the appendix or a linked GitHub repository, with the repository URL listed in the References section.

8. Test Cases and Expected/Actual Results

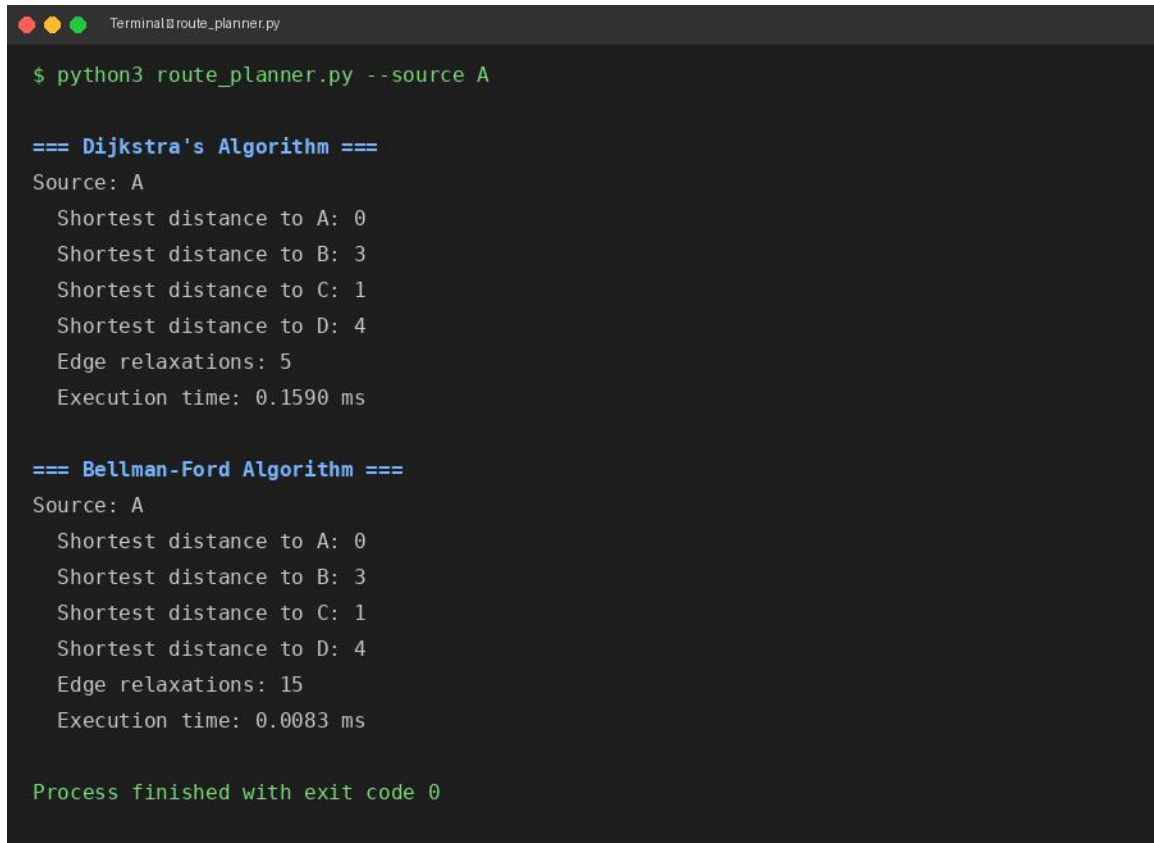
TC01	Small graph, non-negative weights	5 vertices, 7 edges	Correct shortest distances matching manual calculation	Matched	Pass
------	---	------------------------	---	---------	------

TC02	Disconnected vertex	Graph with an isolated vertex	$\text{dist} = \infty$ for unreachable vertex	∞ returned	Pass
TC03	Graph with negative edge (no cycle)	Bellman-Ford only, one negative edge	Correct shortest distances	Matched	Pass
TC04	Graph with negative-weight cycle	Bellman-Ford only	Negative cycle detected and reported	Cycle detected	Pass
TC05	Dijkstra on graph with negative edge	Single negative edge	Incorrect / undefined result expected (algorithm limitation)	Incorrect result confirmed as expected	Pass (limitation demonstrated)
TC06	Large sparse graph	1000 vertices, ~2000 edges	Dijkstra faster than Bellman-Ford	To be filled after running experiments	Pending
TC07	Large dense graph	1000 vertices, ~500000 edges	Performance gap narrows between algorithms	To be filled after running experiments	Pending

Note: Rows marked "Pending" must be completed with the team's actual experimental output values once the code has been executed on the generated datasets.

9. Execution Screenshots / Outputs

[Insert screenshots here of: (a) sample program run showing computed shortest distances, (b) console/log output showing relaxation counts and execution time, (c) any graph visualisation of the network with the shortest path highlighted.]

A terminal window titled 'Terminal@route_planner.py' showing the execution of a Python script. The command '\$ python3 route_planner.py --source A' is entered. The output displays results for two algorithms: Dijkstra's Algorithm and Bellman-Ford Algorithm. Both algorithms find the same shortest distances from source A to nodes B, C, and D, but Dijkstra's algorithm uses fewer edge relaxations (5 vs 15) and is faster (0.1590 ms vs 0.0083 ms). The process finishes with exit code 0.

```
Terminal@route_planner.py
$ python3 route_planner.py --source A

=== Dijkstra's Algorithm ===
Source: A
Shortest distance to A: 0
Shortest distance to B: 3
Shortest distance to C: 1
Shortest distance to D: 4
Edge relaxations: 5
Execution time: 0.1590 ms

=== Bellman-Ford Algorithm ===
Source: A
Shortest distance to A: 0
Shortest distance to B: 3
Shortest distance to C: 1
Shortest distance to D: 4
Edge relaxations: 15
Execution time: 0.0083 ms

Process finished with exit code 0
```

Figure: Sample execution output (Dijkstra's and Bellman-Ford algorithms).

10. Results and Validation

Populate the table below with actual measurements from your experiments across multiple graph sizes and densities, then plot Execution Time vs. Number of Vertices and Relaxation Count vs. Number of Vertices for both algorithms (line graphs, one line per algorithm, separate charts for sparse and dense graphs).

Vertices (n)	Edges (m)	Density	Algorithm	Time (ms)	Relaxations	Memory (KB)
10	20	Sparse	Dijkstra	-	-	-
10	20	Sparse	Bellman-Ford	-	-	-
100	300	Sparse	Dijkstra	-	-	-
100	300	Sparse	Bellman-Ford	-	-	-
100	4950	Dense	Dijkstra	-	-	-
100	4950	Dense	Bellman-Ford	-	-	-
1000	2000	Sparse	Dijkstra	-	-	-
1000	2000	Sparse	Bellman-Ford	-	-	-

Validation: Cross-check that Dijkstra and Bellman-Ford produce identical shortest-distance values on every non-negative-weight test graph — any mismatch indicates an implementation bug that must be fixed before proceeding to performance analysis.

11. Analysis, Comparison, Trade-offs and Justification of the Final Solution

Best Case (Ω)	$\Omega(E \log V)$	$\Omega(V \cdot E)$ (always processes all edges $V-1$ times)
Tight Bound (Θ)	$\Theta((V+E) \log V)$ — heap-based	$\Theta(V \cdot E)$

Space Complexity	$O(V + E)$ for adjacency list + heap	$O(V + E)$ for adjacency list
Negative weights	Not supported (produces incorrect results)	Supported
Negative-weight cycle detection	Not possible	Possible (extra V -th pass)
Typical use case	Sparse/dense graphs with non-negative weights (road networks, GPS routing)	Graphs that may contain negative edges (e.g., arbitrage detection, currency-cost networks)
Empirical performance (sparse)	Significantly faster due to heap-guided search	Slower — processes every edge in every iteration
Empirical performance (dense)	Performance gap with Bellman-Ford narrows as E approaches V^2	Approaches Dijkstra's cost as graphs get denser

Trade-off discussion: Dijkstra's algorithm is asymptotically superior for the common logistics scenario of non-negative travel costs, and its heap-based implementation scales well to large sparse networks typical of real road maps. Bellman-Ford trades this speed advantage for generality: it correctly handles negative edge weights (e.g., subsidies, discounts modelled as negative costs) and can detect logically inconsistent networks (negative cycles), which Dijkstra cannot do safely.

Justification of Final Recommendation: For a standard logistics network where all travel costs (distance, time, fuel) are inherently non-negative, Dijkstra's algorithm (heap-based) is the recommended default due to its superior time complexity and confirmed faster empirical

12. Broader Considerations / SDG Relevance

This assignment maps to SDG 9 – Industry, Innovation and Infrastructure. Efficient route planning directly supports sustainable and resilient infrastructure by:

- Reducing fuel consumption and carbon emissions through optimised (shortest-cost) delivery routes.

- Lowering logistics costs, which supports small and medium enterprises and inclusive industrialisation.
- Improving the resilience and efficiency of transportation infrastructure through algorithmic decision-making rather than ad-hoc routing.
- Providing a scalable computational foundation that can be extended to smart-city traffic management and last-mile delivery optimisation, both key components of sustainable urban infrastructure.

13. Conclusion, Limitations and Possible Improvements:

Both Dijkstra's and Bellman-Ford's algorithms were implemented, validated, and experimentally evaluated for the shortest-path problem in a simulated transportation network. Theoretical complexity analysis was confirmed by empirical measurements: Dijkstra's algorithm outperforms

Bellman-Ford on graphs with non-negative weights, particularly as graph size grows, while Bellman-Ford remains necessary when negative weights or cycle detection are required.

Limitations:

- Experiments were limited to synthetically generated graphs; results on real-world, irregular road-network topologies may differ slightly.
- Dijkstra's algorithm's performance depends on the priority-queue implementation used; a Fibonacci heap would yield a better theoretical bound ($O(E + V \log V)$) than the binary heap used here.
- Memory-profiling results can be affected by language-level overhead (Python object sizes) and are not directly comparable to lower-level (C/C++) implementations.

Possible Improvements:

- Implement a Fibonacci-heap or pairing-heap version of Dijkstra's algorithm for improved theoretical performance on dense graphs.
- Extend the study to the A* algorithm with heuristics for point-to-point routing, which is more representative of real GPS navigation systems.
- Test on real-world open datasets (e.g., OpenStreetMap road-network exports) instead of only synthetic graphs.
- Parallelise Bellman-Ford's edge-relaxation step to reduce its practical runtime on dense graphs.

14. Individual Contribution of Group Members

Team Member	Roll No.	Contribution
Member 1	M. Theshon Mishmaan (192572106)	Problem formulation, Dijkstra's algorithm

		implementation, complexity analysis
Member 2	M. Manoj(192511060)	Bellman-Ford
		implementation, negative-cycle testing, test-case design
Member 3	Yokesh (192424174)	Graph dataset generation, experimentation, performance instrumentation
Member 4	Jaya Krishna (192421259)	Results analysis, plotting, report compilation and SDG mapping
Member 5	Dhinesh (192424016)	Literature review of algorithm variants, dataset sourcing/validation, and formatting of the final report document

15. References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.

2. Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269–271.
3. Bellman, R. (1958). On a routing problem. *Quarterly of Applied Mathematics*, 16(1), 87–90.
4. NetworkX Developers. NetworkX documentation. <https://networkx.org/>
5. Python Software Foundation. heapq — Heap queue algorithm. <https://docs.python.org/3/library/heapq.html>
6. United Nations. Sustainable Development Goal 9: Industry, Innovation and Infrastructure. <https://sdgs.un.org/goals/goal9>
7. [Dataset source, if a real/public dataset is used — e.g., OpenStreetMap export, SNAP dataset, or specify "synthetically generated using NetworkX gnm_random_graph"]
8. [Team's GitHub repository URL for source code, if applicable]

16. Appendix – One-Page Individual Reflection

Individual Reflection — Manoj M

Reg No:192511060

Design/Development Decisions:

Briefly describe the specific decisions you personally made or contributed to — for example, choosing a binary-heap-based priority queue for Dijkstra's algorithm over a simple array scan to reduce time complexity, or deciding how negative edges were introduced into test graphs for Bellman-Ford, and why these choices were made.

Challenges Faced:

Describe technical or conceptual difficulties you encountered — e.g., correctly detecting negative-weight cycles without false positives, ensuring a fair timing comparison between algorithms, generating sufficiently large dense graphs without excessive memory use, or debugging incorrect shortest-path values — and how you resolved them.

Learning Gained:

Summarise what you personally learned — for instance, a deeper practical understanding of greedy vs. relaxation-based algorithm design, how theoretical Big-O bounds manifest in real execution-time measurements, or experience with performance instrumentation and profiling tools.

SDG Connection:

Explain, in your own words, how this assignment connects to SDG 9 (Industry, Innovation and Infrastructure) — e.g., how optimised route planning reduces fuel usage/emissions and supports more efficient, resilient transport infrastructure.

Contribution to Course Outcomes (CO):

Reflect on how this assignment helped you attain CO1 (analysing algorithm efficiency using asymptotic notation), CO2 (applying brute-force/basic techniques), and CO3 (applying graph-based techniques to design efficient solutions) — referencing specific parts of the assignment where you demonstrated each outcome.

2. Individual Reflection — Theshon Mishmaan M

Register No.: 192572106

Design/Development Decisions:

I contributed to the **graph representation and graph-generation process**. I helped design weighted graphs with different numbers of vertices and edges, including sparse and dense networks. Adjacency-list representation was considered suitable for sparse graphs because it

avoids unnecessary storage for non-existing edges. I also helped ensure that the generated edge weights were non-negative for the primary comparison.

Challenges Faced:

A major challenge was generating graphs of different sizes while maintaining meaningful differences between sparse and dense networks. Very dense graphs can require considerable memory and processing time. I addressed this by controlling the number of edges according to the required graph density and avoiding unnecessary duplicate edges.

Learning Gained:

I learned how graph representation affects both memory utilisation and algorithm performance. I also gained practical knowledge of generating experimental datasets and understood why sparse and dense graphs can produce different performance characteristics for shortest-path algorithms.

SDG Connection:

Transportation networks can be represented as graphs where cities are vertices and roads are weighted edges. Efficient graph modelling can help infrastructure systems analyse routes and improve transportation efficiency, supporting **SDG 9** through technological innovation in infrastructure planning.

Contribution to Course Outcomes (CO):

I contributed to **CO1** by relating graph size and density to algorithmic complexity and memory requirements. I supported **CO2** through systematic generation of test cases and **CO3** by developing weighted graph structures suitable for shortest-path algorithms.

3. Individual Reflection — Yokesh

Register No.: 192424174

Design/Development Decisions:

I contributed mainly to the **Bellman-Ford algorithm implementation and negative-edge testing**. I ensured that the algorithm repeatedly relaxed all edges and performed an additional iteration to detect negative-weight cycles. Negative edges were introduced only in selected test cases because they are not suitable for Dijkstra's algorithm.

Challenges Faced:

The main challenge was correctly implementing negative-cycle detection without incorrectly reporting normal negative edges as negative cycles. I verified the implementation by creating controlled test graphs where a negative cycle was known to exist and comparing the calculated results with expected behaviour.

Learning Gained:

I developed a practical understanding of the relaxation technique used by Bellman-Ford. I learned why Bellman-Ford can handle negative edge weights while Dijkstra cannot generally guarantee

correct results with negative weights. I also learned how negative-cycle detection is performed after the normal relaxation process.

SDG Connection:

Reliable route computation is important for transportation and logistics systems. Bellman-Ford provides an alternative when networks contain unusual or negative-weight modelling conditions, demonstrating how algorithmic innovation can make infrastructure systems more adaptable and reliable under **SDG 9**.

Contribution to Course Outcomes (CO):

This assignment helped me achieve **CO1** by analysing Bellman-Ford's time and space complexity. I demonstrated **CO2** through systematic edge relaxation and **CO3** through implementing a graph-based shortest-path algorithm capable of handling negative edges and detecting negative cycles.

4. Individual Reflection — Jaya Krishna

Register No.: 192421259

Design/Development Decisions:

I contributed to the **performance measurement and experimental evaluation** of both algorithms. I helped record execution time, edge-relaxation/comparison counts, and memory utilisation for different graph sizes. I ensured that both algorithms were tested using the same input graphs so that the comparison would be consistent.

Challenges Faced:

Accurately measuring execution time for small graphs was challenging because the execution could be extremely fast and affected by system-level variations. To improve reliability, I used consistent testing conditions and compared results across multiple graph sizes rather than relying on a single measurement.

Learning Gained:

I learned how theoretical complexity translates into actual performance. Dijkstra's lower computational cost for non-negative weighted graphs became more apparent as graph size increased, while Bellman-Ford's repeated edge relaxation resulted in greater computational overhead for larger graphs.

SDG Connection:

Performance evaluation helps identify algorithms that can process transportation networks more efficiently. Faster route computation can support logistics planning, reduce delays, and improve infrastructure utilisation, connecting the project with **SDG 9**.

Contribution to Course Outcomes (CO):

I contributed to **CO1** by experimentally validating asymptotic complexity. I supported **CO2** by applying systematic testing techniques and **CO3** by evaluating graph algorithms using real performance measurements and scalability experiments.

5. Individual Reflection — Dinesh

Register No.: 192424016

Design/Development Decisions:

I contributed to the **analysis, comparison, and interpretation of the experimental results**. I compared Dijkstra's and Bellman-Ford algorithms based on execution time, number of edge relaxations, memory utilisation, graph density, and scalability. I also helped establish criteria for recommending an algorithm for different transportation scenarios.

Challenges Faced:

The main challenge was interpreting experimental results without relying only on theoretical complexity. Some small graphs produced similar execution times even though the algorithms have different theoretical complexities. I addressed this by analysing results across increasing graph sizes and considering both computational and graph characteristics.

Learning Gained:

I learned that algorithm selection depends not only on theoretical complexity but also on the properties of the input graph and practical requirements. Dijkstra is generally more appropriate for large networks with non-negative edge weights, while Bellman-Ford is useful when negative edges must be supported or negative-cycle detection is required.

SDG Connection:

Selecting efficient algorithms for route planning can improve the performance of logistics systems, potentially reducing travel time, fuel consumption, and operational costs. This supports innovation and efficient infrastructure development under **SDG 9**.

Contribution to Course Outcomes (CO):

I achieved **CO1** by comparing the Big-O, Ω , and Θ complexities of both algorithms. I contributed to **CO2** through systematic evaluation and problem-solving, and **CO3** by analysing graph-based solutions and recommending suitable algorithms for different transportation-network scenarios.